Highlights

Bensemble: A Python Library for Bayesian Ensembling

Fedor Sobolevsky, Dmitrii Vasilenko, Mukhammadsharif Nabiev, Vadim Kasyuk

- A unified framework for Bayesian ensembling methods, PyTorch-compatible.
- Diversity of implemented ensembling methods, including variational inference, probabilistic backpropagation, neural ensemble search and more.
- Evaluated on several benchmarks to ensure functionality and compare implemented methods.

Bensemble: A Python Library for Bayesian Ensembling

Fedor Sobolevsky[a,b], Dmitrii Vasilenko[a], Mukhammadsharif Nabiev[a] and Vadim Kasyuk[a]

[a]Moscow Institute of Physics and Technology, Moscow, Russia
[b]Moscow State University, Moscow, Russia

ARTICLE INFO

Keywords:
Bayesian ensembling
uncertainty
PyTorch
Python

ABSTRACT

In this technical report, we introduce Bensemble, a Python library designed for approximating the posterior distribution of neural network weights to facilitate Bayesian ensembling. Standard deep learning models often yield overconfident predictions, since they typically don't take into account the uncertainty which comes with model training. Our library addresses this by providing a unified, PyTorch- and Lightning-compatible interface for a variety of Bayesian ensembling methods, including variational inference, probabilistic backpropagation, Laplace approximation, neural ensemble search and others. Bensemble operates on the principle that stochastic weight distributions act as generators for diverse model ensembles. It decouples the inference phase (learning an approximation of the posterior) from the prediction phase (sampling deterministic members for Bayesian model averaging). We provide a detailed description of Bensemble's interface and implemented methods. We also demonstrate the library's capabilities on benchmarks focusing on calibration and adversarial robustness.

1. Introduction

Standard neural networks typically optimize a point estimate of their weights, which fails to capture epistemic uncertainty. Bayesian ensembling methods offer an alternative approach by approximating a probability distribution over model weights. This enables uncertainty estimation by averaging model predictions over the weight posterior. While exact inference is intractable, approximate methods allow us to learn a surrogate distribution over model weights. This approximated posterior can be treated as a source from which we sample an ensemble of deterministic models. Prediction and uncertainty estimation can then be done by aggregating predictions across this ensemble.

Currently, there exists a wide variety of methods implementing different solutions to the problem of weight posterior approximation. These methods use different techniques and training/post-training procedure, and so far their implementations have been scattered across multiple interfaces and libraries. To solve this issue, we propose Bensemble, a library that implements a wide variety of Bayesian ensembling methods and standardizes the workflow of probabilistic deep learning.

Our work makes the following main contributions:

1. We introduce Bensemble, a Python library with a unified API that abstracts the complexity of seven different Bayesian ensembling methods and their variations, allowing for standardized usage and convenient method comparison.
2. We provide a variety of helper methods in Bensemble useful for probabilistic deep learning and model ensembling, as well as PyTorch and Lightning compatibility.
3. We conduct extensive testing of all of Bensemble's methods to ensure quality implementations and provide sensible defaults for hyperparameters derived from testing.

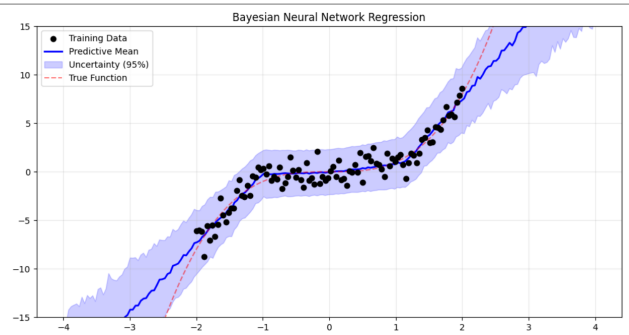

Figure 1: An example of regression with uncertainty using VI

2. Methodology

2.1. Problem Statement

Bensemble's core objective is weight posterior approximation for neural networks. Formally, given a dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$ and a neural network with weights w, we seek the posterior distribution

$$p(w \mid \mathcal{D}) \propto p(\mathcal{D} \mid w)p(w).$$

Since the exact posterior is intractable in the case of most neural networks, we apply a variety of different methods to approximate the true posterior with a parametric distribution $q_\theta(w)$. From this distribution, we can sample various sets of model weights to produce an ensemble of models f_{w_i}. Predictions are then made by averaging predictions of each member model from the ensemble:

$$\hat{y} = \sum_{i=1}^{K} f_{w_i}(x).$$

Uncertainty of predictions can be then defined as the standard deviation of member predictions (see figure 1).

2.2. Implemented methods

We will now briefly describe the methods implemented as part of Bensemble.

Following Gal and Ghahramani (2016), we implement **Monte-Carlo dropout (MCD)** as a Bayesian ensembling method. For a neural network trained with dropout, this method works by leaving dropout on at the inference stage. This way, we get an ensemble of models, each with random weight subsamples from the original model.

We adopt the **practical variational inference (PVI)** framework proposed by Graves (2011), approximating the posterior with a diagonal Gaussian $q_\theta(w)$. The objective is to minimize the variational free energy, often interpreted as a Minimum Description Length (MDL) cost:

$$\mathcal{L}(\theta) = \underbrace{\mathbb{E}_{q_\theta(w)}[\mathcal{L}_\mathcal{D}(w)]}_{\text{error cost}} + \underbrace{\mathrm{KL}(q_\theta(w) \| p(w))}_{\text{complexity cost}},$$

where $\mathcal{L}_\mathcal{D}$ is the negative log-likelihood.

To optimize the error cost efficiently, we employ the local reparameterization trick (LRT) Kingma, Salimans and Welling (2015). As noted by Graves (2011), sampling weights directly introduces high variance in gradient estimates. LRT addresses this by sampling pre-activations instead. For a linear layer with inputs X, weight means M, and variances V, the pre-activation Γ is distributed as:

$$\Gamma \sim \mathcal{N}(XM^T, X^2V^T).$$

We sample $\zeta = XM^T + \varepsilon \odot \sqrt{X^2V^T}$, where $\varepsilon \sim \mathcal{N}(0, I)$, allowing for stable, low-variance backpropagation. The complexity cost is computed analytically as the sum of KL divergences for each weight.

For **variational inference with Rényi divergence (VR)**, introduced by Li and Turner (2016), we implement a weight perturbation approach. Unlike our VI implementation, here, explicit weights $w \sim \mathcal{N}(\mu, \mathrm{softplus}(\rho))$ are sampled during the forward pass. The objective generalizes the ELBO using α-Rényi divergence:

$$\mathcal{L}_{\mathrm{VR}}(\theta, \alpha) = -\frac{1}{1-\alpha} \log \frac{1}{K} \sum_{k=1}^{K} \left(\frac{p(\mathcal{D}, w_k)}{q_\theta(w_k)} \right)^{1-\alpha}.$$

The parameter α (default 1.0) controls the bias-variance trade-off.

Our implementation of **probabilistic backpropagation (PBP)** by Hernández-Lobato and Adams (2015) uses assumed density filtering (ADF) in an online fashion. Means and variances are propagated analytically. For ReLU activations, we use exact moment matching functions relying on the PDF/CDF of the standard normal. Weights are updated by matching the moments of the tilted distribution $q(w)p(y|x, w)$. We compute gradients of the log-partition function $\log Z$ to update μ and Σ directly, bypassing standard SGD.

We also implement a scalable **Laplace approximation (LA)** using Kronecker-factored approximate curvature as described in Ritter, Botev and Barber (2018). Since this is, unlike others, a post-hoc posterior approximation method, a standard network is first trained to find the mode W_l^{MAP}. We then capture covariance of activations (A_l) and pre-activation gradients (G_l). The posterior for layer l is then approximated as a matrix normal distribution $\mathcal{MN}(W_l^{\mathrm{MAP}}, A_l, G_l)$. Samples are generated efficiently using eigen-decomposition of Kronecker factors:

$$W_{\text{sample}} = W_{\mathrm{MAP}} + L_V Z L_U^T, \tag{1}$$

where L_V, L_U are Cholesky factors of the inverse regularized covariances and Z is sampled from the standard matrix normal distribution.

Neural ensemble search (NES), introduced by Zaidi, Zela, Elsken, Holmes, Hutter and Teh (2021), employs neural architecture search in order to find an optimal ensemble of models with varying architectures. Formally, the method searches through model architectures α_i in a predefined architecture search space $\mathcal{A}$. To find the optimal ensemble, NES solves the following bilevel optimization problem:

$$\min_{\alpha_1,\ldots,\alpha_M \in \mathcal{A}} \mathcal{L}\Big(\mathrm{Ensemble}(f_{\theta_1,\alpha_1}, \ldots, f_{\theta_M,\alpha_M}),\ \mathcal{D}_{\mathrm{val}}\Big)$$

$$\text{s.t.} \quad \theta_i \in \arg\min_\theta \mathcal{L}(f_{\theta,\alpha_i}, \mathcal{D}_{\mathrm{train}}), \quad i = 1, \ldots, M$$

For architecture searching, we implement both strategies described in Zaidi et al. (2021) — random searching (NES-RS) and regularized evolution (NES-RE). While the first strategy relies on a simple random searching procedure, the second one utilizes an evolutionary algorithm.

Finally, we also implement **neural ensemble search via Bayesian sampling (NESBS)**, introduced by Shu, Chen, Dai and Low (2022), as an alternative to NES that aims to reduce the computational cost of NES. To do so, it utilizes training a supernet with uniform path sampling to share weights across different model architectures. Then, a variational posterior over architectures is introduced:

$$p_\alpha(\mathcal{A}) \approx p(\mathcal{A}|\mathcal{D}).$$

The variational posterior is learned via evidence lower bound (ELBO) minimization:

$$\max_\alpha \mathbb{E}_{\mathcal{A}\sim p_\alpha(\mathcal{A})}[\log p(\mathcal{A}|\mathcal{D})] - KL[p_\alpha(\mathcal{A})|p(\mathcal{A})].$$

Ensemble member architectures can then be sampled from the variational posterior via simple Monte-Carlo sampling or using Stein variational gradient descent with regularized diversity (SVGD-RD), i. e. via controlled optimization of the set of architectures with the following objective:

$$q^* = \arg\min_{q\in\mathcal{Q}} \mathrm{KL}(q\|p) + n\delta\mathbb{E}_{x,x'\sim q}[k(x, x')].$$

3. Implementation Details

The architecture of Bensemble is built around the 'Ensemble' class, incapsulating an ensemble of models or a stochastic ensemble (i.. an ensemble where model weights

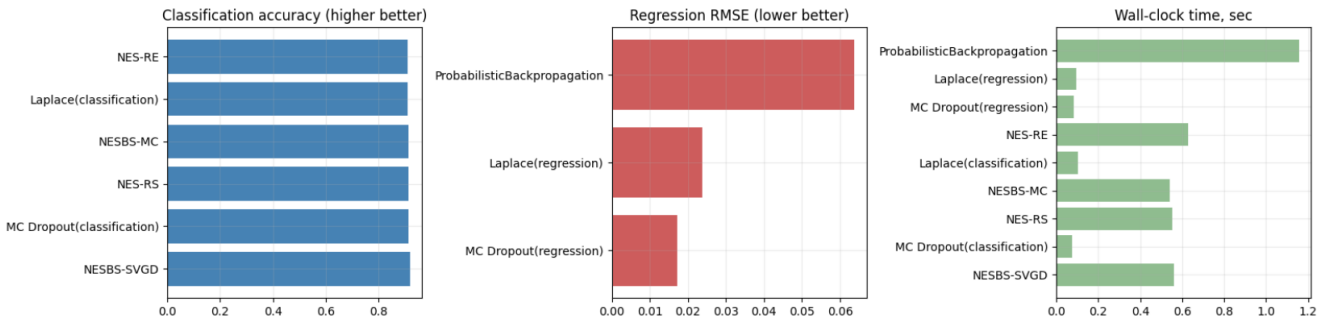

Figure 2: Results on synthetic dataset

are sampled in an online fashion). Each of Bensemble's methods can be used to produce an instance of this base class, which then can be used to make predictions with uncertainty.

MCD, VR, VI and PBP are all implemented as stochastic ensemble generators. Notably, while we implement most of the methods as self-contained classes, VI is implemented via a Bayesian linear layer class. LA and NES variants are implemented as generators of model ensembles. We implement neural ensemble search algorithm via integration with the NNI library Microsoft (2021).

We also implement a variety of utilities for Bayesian machine learning in Bensemble, including but not limited to: uncertainty decomposition (into aleatoric and epistemic components), evaluation metrics (NLL, ECE, Brier score) and calibration for classification probabilities (temperature scaling and vector scaling). The library's full code is available at `github.com/intsystems/bensemble`.

4. Experimental Evaluation

4.1. Experimental setup

We evaluated the library on the Boston Housing dataset Harrison Jr and Rubinfeld (1978), as well as synthetic classification and regression datasets. The methods were applied to an MLP with a fixed architecture made of two hidden layers with size 50 and ReLU activations. MCD was used with adropout rate of 0.2. For PBP, we use an inverse Gamma prior with noise shape parameters $\alpha = 6.0, \beta = 6.0$. For Laplace, we perform MAP training with learning rate $\alpha = 10^{-3}$ and K-FAC estimated on the full training set. VI and VR were tested with priors with $\sigma = 1.0$, initialized by $\rho = -2.0$ (approximately $\sigma = 0.12$). For NES, we used a simple search space with varying hidden layer counts, sizes and varying dropout rates. NESBS was used with temperature 0.8, 8 SVGD steps and a learning rate of 0.25 for SVGD.

4.2. Synthetic data

Figure 2 presents the results of evaluation on synthetic classification and regression datasets. We can see that, overall, all methods perform well, with some exceeding others in accuracy and some leading in time performance.

Table 1
Performance on Boston Housing (Test Set). Lower is better.

Method	RMSE	NLL	ECE	Brier
Deterministic MLP	4.21	47.99	0.033	0.090
MCD	**3.84**	3.68	0.019	**0.055**
VI	4.63	**2.88**	0.008	0.099
PBP	5.42	3.17	0.008	0.091
VR	5.36	3.16	**0.006**	0.127
LA	19.38	4.55	0.011	0.241

Table 2
RMSE under FGSM adversarial attacks. **Bold** indicates best performance.

Method	0.00	0.10	0.30	0.50	1.00
Det. MLP	4.21	7.41	13.13	17.70	27.74
MCD	**3.80**	**5.16**	8.82	12.67	23.05
VI	4.58	**5.16**	**6.40**	**7.71**	**12.22**
PBP	5.42	5.73	6.87	8.28	12.34
VR	5.35	5.76	6.64	8.74	13.50
LA	19.22	24.04	22.92	26.40	40.46

4.3. Boston Housing dataset

Table 1 shows the results of evaluation on the Boston Housing dataset. We can see that while deterministic models are accurate (low RMSE), they are poorly calibrated (high NLL). PBP and VI achieve the best balance of accuracy and calibration.

4.4. Adversarial Robustness

We subjected models to FGSM attacks (Table 2). The deterministic model fails catastrophically at $\varepsilon = 0.1$. VI and PBP demonstrate superior robustness. Specifically, PBP's moment matching effectively filters high-frequency adversarial noise, maintaining low error rates even at high perturbation levels.

5. Conclusion

We presented Bensemble, a library that democratizes access to stochastic weight modeling. We implemented a wide variety of Bayesian ensembling methods, applicable

to different tasks and in various usage scenarios. By validating our implementations of these methods on a number of benchmarks, we ensure that Bensemble provides a robust toolkit for uncertainty estimation. Our experiments confirm that Bayesian ensembling noticeably improves calibration and adversarial robustness compared to standard point estimates.

References

Gal, Y., Ghahramani, Z., 2016. Dropout as a bayesian approximation: Representing model uncertainty in deep learning, in: international conference on machine learning, PMLR. pp. 1050–1059.

Graves, A., 2011. Practical variational inference for neural networks. Advances in neural information processing systems 24.

Harrison Jr, D., Rubinfeld, D.L., 1978. Hedonic housing prices and the demand for clean air. Journal of environmental economics and management 5, 81–102.

Hernández-Lobato, J.M., Adams, R., 2015. Probabilistic backpropagation for scalable learning of bayesian neural networks, in: International conference on machine learning, PMLR. pp. 1861–1869.

Kingma, D.P., Salimans, T., Welling, M., 2015. Variational dropout and the local reparameterization trick. Advances in neural information processing systems 28.

Li, Y., Turner, R.E., 2016. Rényi divergence variational inference. Advances in neural information processing systems 29.

Microsoft, 2021. Neural Network Intelligence. URL: https://github.com/microsoft/nni.

Ritter, H., Botev, A., Barber, D., 2018. A scalable laplace approximation for neural networks, in: International conference on learning representations.

Shu, Y., Chen, Y., Dai, Z., Low, B.K.H., 2022. Neural ensemble search via bayesian sampling, in: Uncertainty in Artificial Intelligence, PMLR. pp. 1803–1812.

Zaidi, S., Zela, A., Elsken, T., Holmes, C.C., Hutter, F., Teh, Y., 2021. Neural ensemble search for uncertainty estimation and dataset shift. Advances in Neural Information Processing Systems 34, 7898–7911.